

Advanced AI and Data Science SoSe 26

DR. TECHN. DI MUGDIM BUBLIN

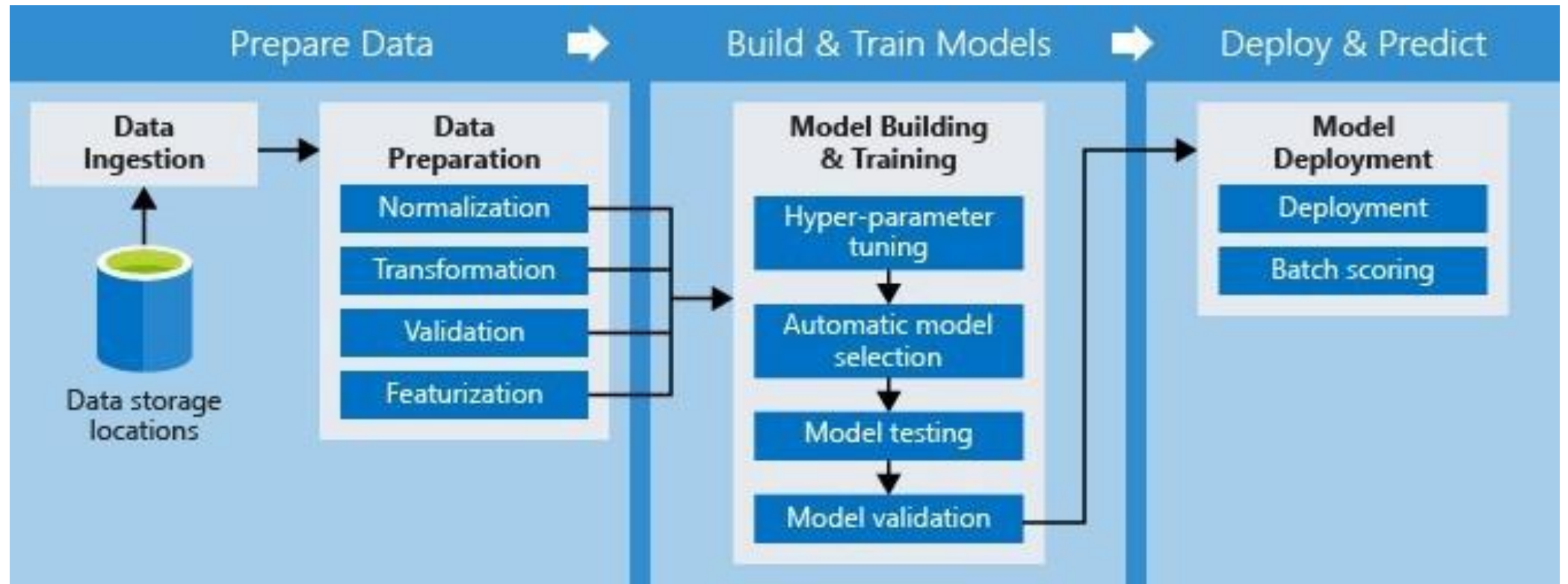
ML Project



Machine Learning Project

1. Frame the problem and look at the big picture.
2. Get the data.
3. Explore the data to gain insights.
4. Prepare the data to better expose the underlying data patterns to Machine Learning algorithms.
5. Build ML pipeline and provide first results fast.
6. Use existing (pre-trained) models as much as possible
7. Explore many different models and shortlist the best ones.
8. Fine-tune your models and combine them into a great solution.
9. Present your solution.
10. Launch, monitor, and maintain your system.

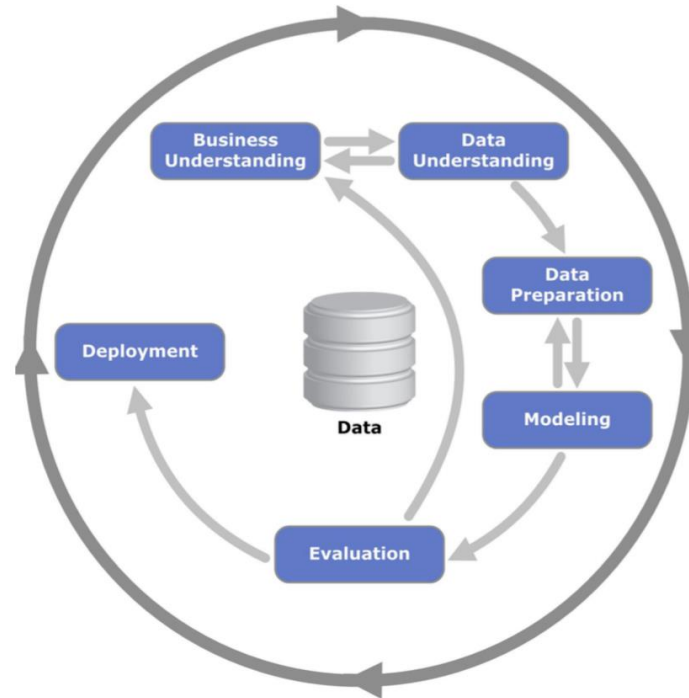
Machine Learning Pipeline



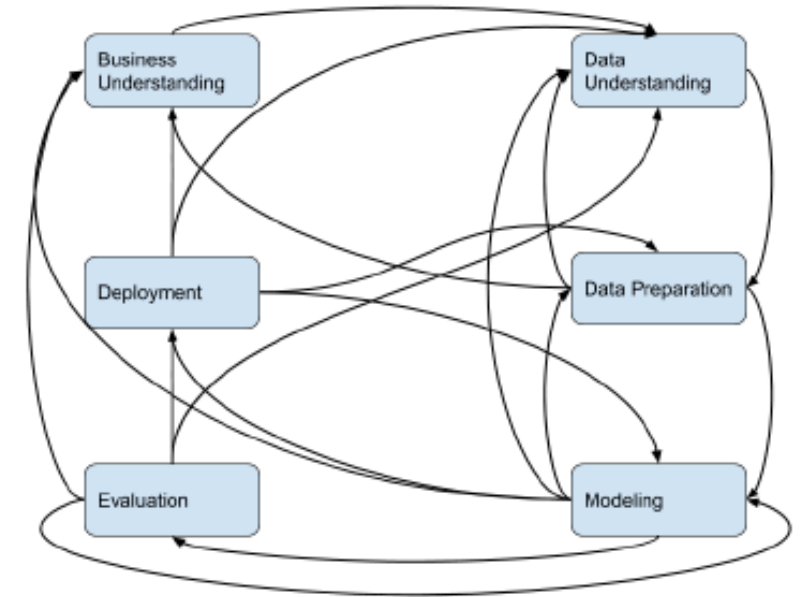
“Clean” ML Design

Avoid chaos by:

- Clean Architecture
- Clean Code
- Domain Understanding



Idealized



Simplified but Realistic ML tornado workflow

from M. Taifi: "Clean Machine Learning Code"

Frame the Problem and Look at the Big Picture

1. Obtain **Domain Knowledge**
2. Which **decisions** should be made?
3. How will your solution be used?
4. What are the **current solutions**/workarounds (if any)?
5. What are comparable problems? Can you reuse experience or tools?
6. Is human expertise available?
7. How would you solve the problem manually?
8. How should you frame this problem (supervised/unsupervised, online/offline, etc.)?
9. How should **performance** be measured?
 - Is the performance measure aligned with the business objective?
 - What would be the minimum performance needed to reach the business objective?
10. List the assumptions you (or others) have made so far.
 - Verify assumptions if possible

Get the Data

1. List the ***data you need and how much*** you need.
2. Find and ***document where*** you can get that data.
3. Check how much ***space*** it will take.
4. Check ***legal obligations***, and get authorization if necessary.
5. Create a ***workspace*** (with enough storage space).
AI Engineering, SDE, SoSe 25
6. Get the data.
7. Convert the data to a ***format*** you can easily manipulate.
8. Ensure sensitive information is deleted or protected (e.g., ***anonymized***).
9. Check the ***size and type of data*** (time series, sample, geographical, etc.).
10. Sample a ***test set, put it aside, and never look at it*** (no data snooping!).

Data Sources

- ***Use a public dataset***
 - Kaggle: <https://www.kaggle.com/datasets>
 - Hugging Face: <https://huggingface.co/datasets>
 - Nicolas Iderhoff: <https://github.com/niderhoff/nlp-datasets>
 - Google's search engine for datasets: <https://datasetsearch.research.google.com/>
- ***Scrape data***
 - Scrape data from Internet and get it labeled by human annotators
- ***Product intervention***
 - AI team should work with the product team to collect more and richer data by developing better instrumentation in the product. In the tech world, this is called *product intervention*.
- ***Data augmentation***

Explore the Data

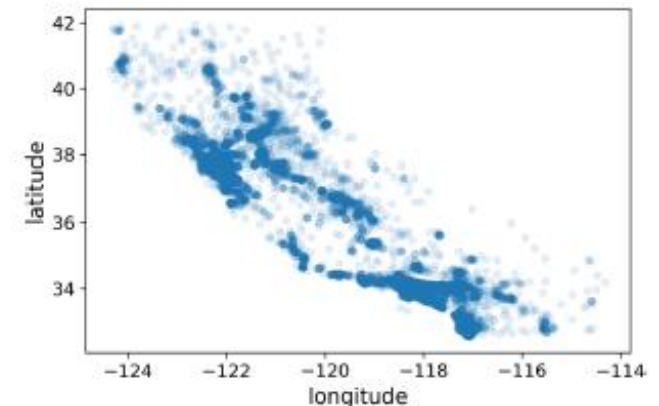
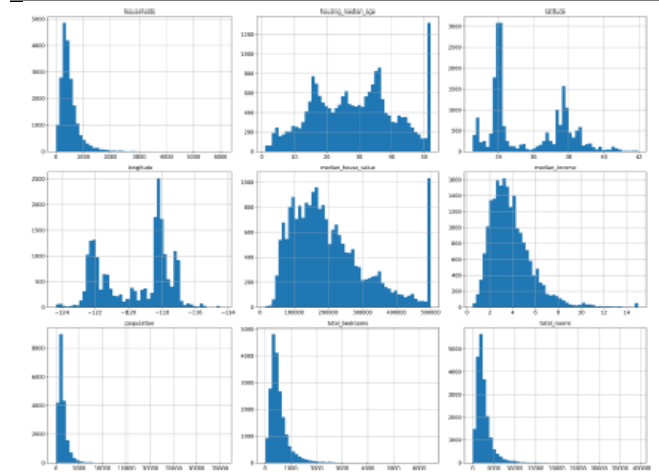
1. Create a copy of the data for exploration (sampling it down to a manageable size if necessary).
2. Study each attribute and its characteristics:
 - Name
 - Type (categorical, int/float, bounded/unbounded, text, structured, etc.)
 - % of missing values
 - Noisiness and type of noise (stochastic, outliers, rounding errors, etc.)
 - Usefulness for the task
3. Type of **distribution** (Gaussian, uniform, logarithmic, etc.)
4. For supervised learning tasks, identify the target attribute(s).
5. **Visualize** the data.
6. Study the **correlations** between attributes

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.687170
total_rooms           0.135231
housing_median_age    0.114220
```
7. Study how you would solve the problem manually.
8. Identify the promising transformations you may want to apply.
9. Identify extra data that would be useful (go back to “Get the Data”).
10. Document what you have learned.

```
In [5]: housing = load_housing_data()
housing.head()
```

Out[5]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0



Prepare the Data

1. Work on copies of the data (keep the original dataset intact).
2. Write functions for all data transformations you apply
3. Data cleaning:
 - Fix or remove outliers (optional).
 - Fill in missing values (e.g., with zero, mean, median...) or drop their rows (or columns).
 - Feature selection
4. Drop the attributes that provide no useful information for the task.
5. Feature engineering, where appropriate:
 - Discretize continuous features.
 - Decompose features (e.g., categorical, date/time, etc.).
 - Add promising transformations of features (e.g., $\log(x)$, $\sqrt{x}$, x , etc.).
 - Aggregate features into promising new features.
6. Feature scaling:
 - Standardize or normalize features.
7. Build Data (Pre)processing pipeline

```
>>> from sklearn.preprocessing import OneHotEncoder
>>> cat_encoder = OneHotEncoder()
>>> housing_cat_1hot = cat_encoder.fit_transform(housing_cat)

>>> housing_cat_1hot.toarray()
array([[1., 0., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.],
       ...,
       [0., 1., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('attribs_adder', CombinedAttributesAdder()),
    ('std_scaler', StandardScaler()),
])

housing_num_tr = num_pipeline.fit_transform(housing_num)
```

Text Extraction and Cleanup Procedures

- HTML Parsing and Cleanup
- Unicode Normalization
- Spelling Correction
- System-Specific Error Correction

[illegible]

```
<p>Book will be around 350 pages. It will be accompanied by a code repository containing several Jupyter notebooks for all the chapters to give a walk-through and explain the code in detail. The code base is in Python and various machine learning and natural language processing libraries. The book assumes that the readers have a good grasp of programming but no theoretical and practical knowledge of NLP.</p>
<p><br>
</p>
<a href="https://www.oreilly.com/library/view/practical-natural-language/9781492054047/">

</a>
<h1 style="color: #e74c3c;">Commonly Asked Questions</h1>
<ul>
<li>Can I contribute to the book?
<p>The book is accompanied by open source Jupyter notebooks and demo applications. If you are a great ML or front-end engineer looking to build something meaningful you can apply by filling <a href="https://goo.gl/forms/d06NcW251lX26ajk1">this form</a>. Also refer to the next question.</p>
```



Supplement Facts

Serving Size 1 Lozenge

Amount Per Serving	% Daily Value
Vitamin B ₁₂ 5000 mcg	208333%
(as methylcobalamin)	

CAUTION: AS WITH ANY DIETARY SUPPLEMENT, CONSULT YOUR HEALTHCARE PRACTITIONER BEFORE USING THIS PRODUCT, ESPECIALLY IF YOU ARE PREGNANT, NURSING, ANTICIPATE SURGERY, TAKE MEDICATION ON A REGULAR BASIS OR ARE OTHERWISE UNDER MEDICAL SUPERVISION.

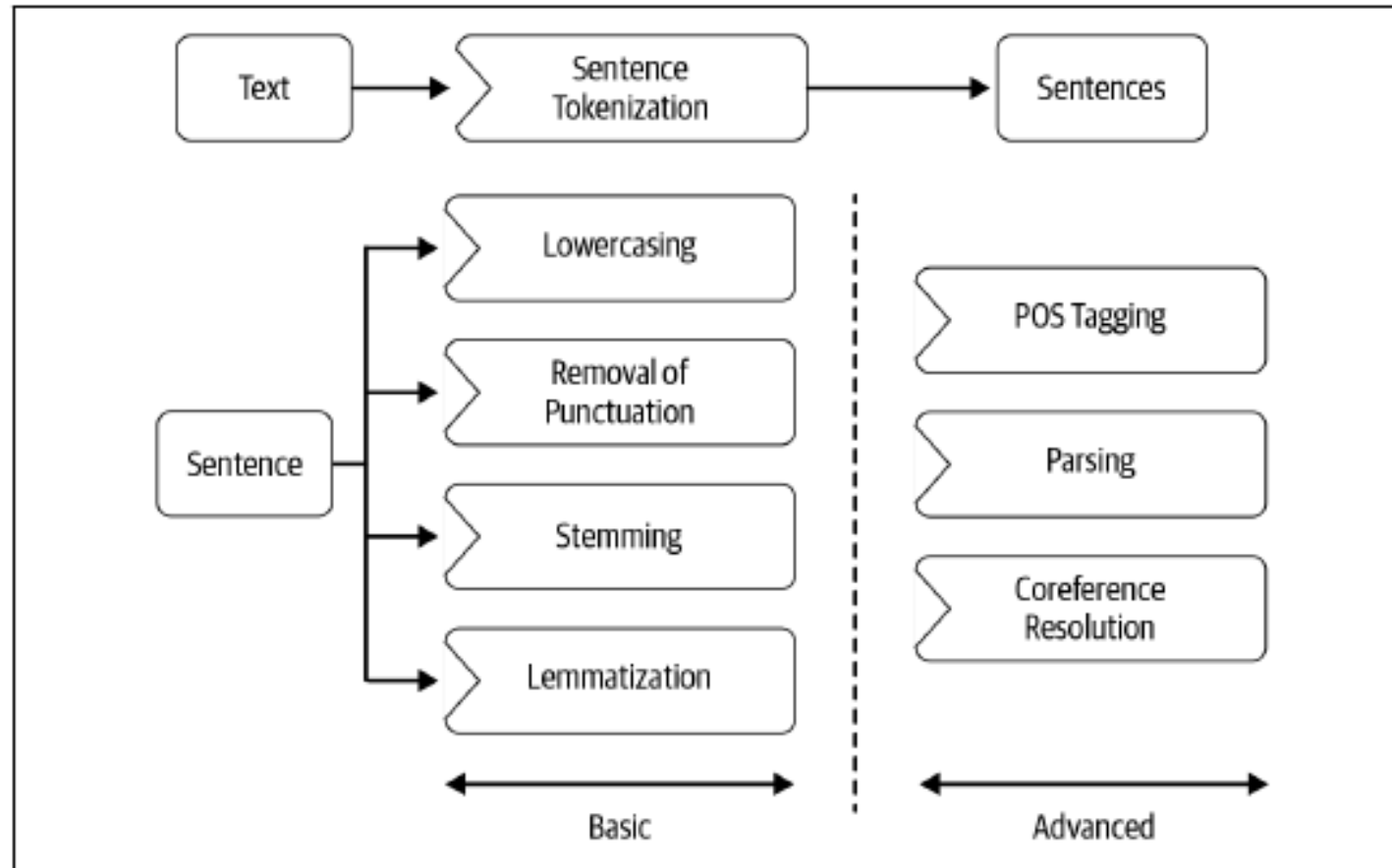
KEEP OUT OF REACH OF CHILDREN. FOR ADULT USE ONLY.

DO NOT USE IF SEAL IS BROKEN OR MISSING.

STORE IN A COOL, DRY PLACE.

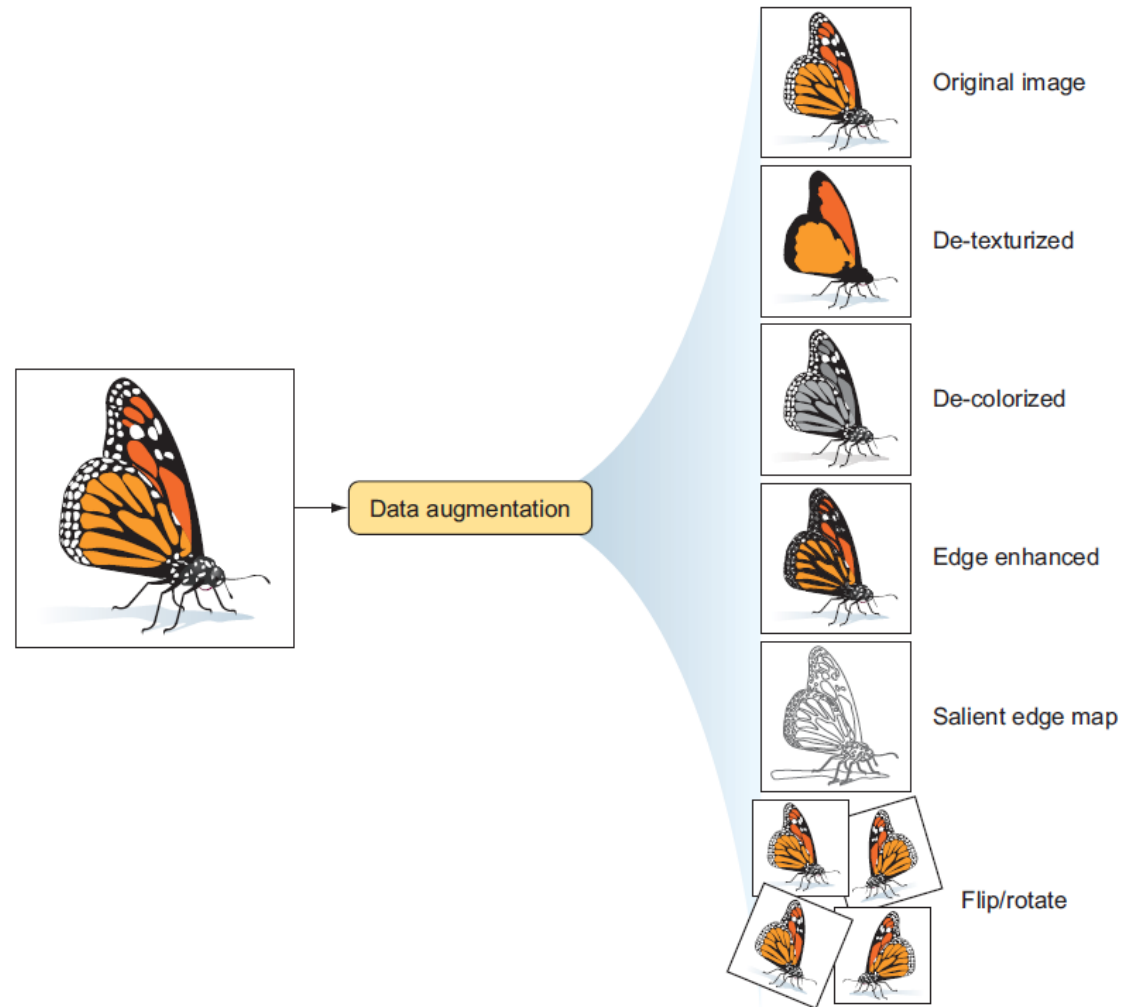
SUGGESTED USE:
TAKE 1 LOZENGE DAILY.

Overview: NLP Pre-Processing Steps



Data Augmentation Computer Vision

- Image-augmentation techniques create modified versions of the input image
- Provide more examples for the ML model to learn from.
- Care about what is allowable transformation – depends on application



Data Augmentation NLP

- **Synonym replacement**

- Randomly choose “k” words in a sentence that are not stop words. Replace these words with their synonyms. For synonyms, we can use Synsets in Wordnet.

- **Back translation**

- Translate text in other language and then back in the same language

- **Bigram flipping**

- For example replace “going to” with “to going”

- **Replacing entities**

- Replace entities like person name, location, organization, etc., with other entities
- in the same category

- **Adding noise to data**

- Add randomly spelling mistakes

Data Division

Divide your data in:

- **Training set** — Which you run your learning algorithm on.
 - **Validation/Development set** — Which you use to tune parameters, select features, and make other decisions regarding the learning algorithm. Sometimes also called the hold-out cross validation set .
 - **Test set** — which you use to evaluate the performance of the algorithm, but not to make any decisions regarding what learning algorithm or parameters to use.
-
- Size of development and test sets should be **according to the performance requirements**: For example for achieving 99.99% accuracy you need at least 10 000 elements in test set.
 - Development and test set should come ***from the same distribution***

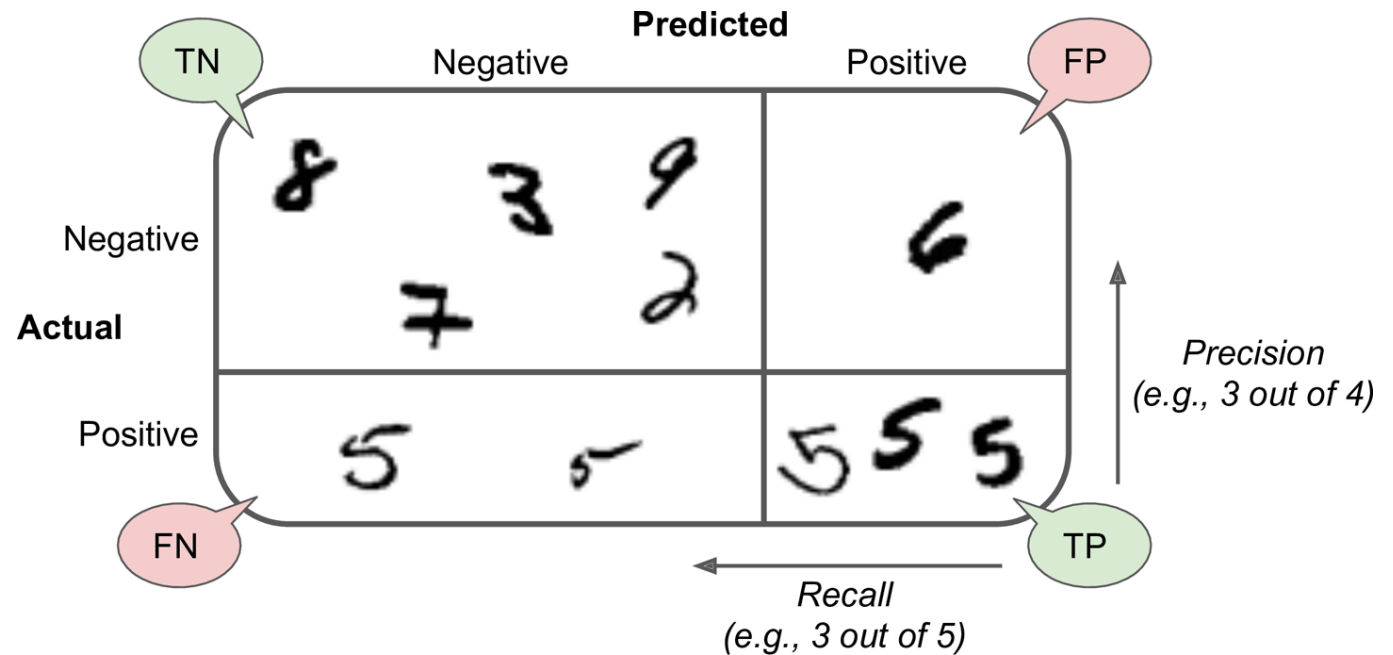
Performance Scores

Task: *Recognize '5'*

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$



TP: True Positive
FP: False Positive
FN: False Negative

Establish Performance Metrics

Establish a **single-number** evaluation metric to optimize!

- **Precision** of a cat classifier is the fraction of images in the dev (or test) set it labeled as cats that really are cats.
- **Recall** is the percentage of all cat images in the dev (or test) set that it correctly labeled as a cat.
- **F1 score** = $2 / ((1/\text{Precision}) + (1/\text{Recall}))$

Optimizing and Satisficing Metrics: Maximize F1 score (optimizing metric) under conditions that running time is below certain level (say 100 ms) and number of false alarms is less than 1 in month (satisficing metrics).

		Predicted	
		+	-
Actual	+	TP	FN
	-	FP	TN

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Performance Metrics for Image Segmentation

- **Intersection over Union (IoU)** — Also known as Jaccard Index, is a measure to describe the extent of overlap of predicted segmentation mask with ground truth. It is defined as the intersection area between the predicted segmentation and the ground truth, divided by the area of union between the predicted segmentation mask and the ground truth:

$$\text{IoU} = J(A, B) = |A \cap B| / (A \cup B) = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$$

where A and B denote the ground truth and the predicted segmentation, respectively, it goes between 0 and 1.

NLP Metrics

- Intrinsic evaluation metrics commonly used to measure performance of NLP systems.
- For most metrics in this category, we assume a test set where we have the ground truth or labels (human annotated, correct answers).
- For some cases, like machine translation or summarization, it's not always possible to automate evaluation since comparison is not objective.

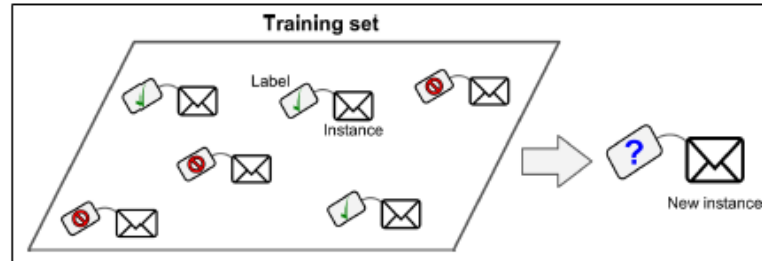
Metric	Description	Applications
AUC [48]	Captures the count of positive predictions that are correct versus the count of positive predictions that are incorrect as we vary the threshold for prediction.	Used to measure the quality of a model independent of the prediction threshold. It is used to find the optimal prediction threshold for a classification task.
MRR (mean reciprocal rank) [50]	Used to evaluate the responses retrieved given their probability of correctness. It is the mean of the reciprocal of the ranks of the retrieved results.	Used heavily in all information-retrieval tasks, including article search, e-commerce search, etc.
MAP (mean average precision) [51]	Used in ranked retrieval results, like MRR. It calculates the mean precision across each retrieved result.	Used in information-retrieval tasks.
RMSE (root mean squared error) [48]	Captures a model's performance in a real-value prediction task. Calculates the square root of the mean of the squared errors for each data point.	Used in conjunction with MAPE in the case of regression problems, from temperature prediction to stock market price prediction.
MAPE (mean absolute percentage error) [52]	Used when the output variable is a continuous variable. It is the average of absolute percentage error for each data point.	Used to test the performance of a regression model. It is often used in conjunction with RMSE.
BLEU (bilingual evaluation understudy) [53]	Captures the amount of n-gram overlap between the output sentence and the reference ground truth sentence. It has many variants.	Mainly used in machine-translation tasks. Recently adapted to other text-generation tasks, such as paraphrase generation and text summarization.
METEOR [54]	A precision-based metric to measure the quality of text generated. It fixes some of the drawbacks of BLEU, such as exact word matching while calculating precision. METEOR allows synonyms and stemmed words to be matched with the reference word.	Mainly used in machine translation.
ROUGE [55]	Another metric to compare quality of generated text with respect to a reference text. As opposed to BLEU, it measures recall.	Since it measures recall, it's mainly used for summarization tasks where it's important to evaluate how many words a model can recall.
Perplexity [56]	A probabilistic measure that captures how confused an NLP model is. It's derived from the cross-entropy in a next word prediction task. The exact definition can be found at [56].	Used to evaluate language models. It can also be used in language-generation tasks, such as dialog generation.

Select Model

Supervised learning algorithms:

- k-Nearest Neighbors
- Linear Regression
- Logistic Regression
- Support Vector Machines (SVMs)
- Decision Trees and Random Forests
- Neural networks

Supervised learning: Classification



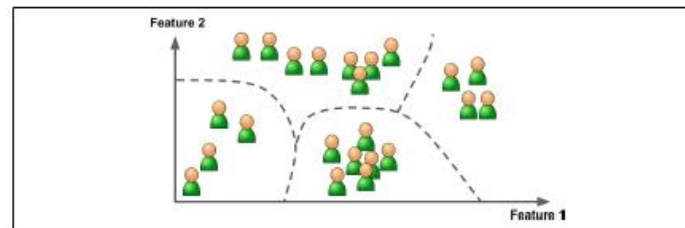
Supervised learning: Regression



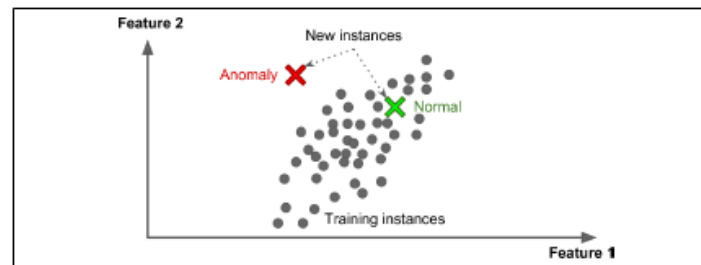
Unsupervised learning algorithms:

- Clustering
 - K-Means
 - DBSCAN
 - Hierarchical Cluster Analysis (HCA)
- Anomaly detection and novelty detection
 - One-class SVM
 - Isolation Forest
- Visualization and dimensionality reduction
 - Principal Component Analysis (PCA)
 - Kernel PCA
 - Locally Linear Embedding (LLE)
 - t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Association rule learning
 - Apriori
 - Eclat

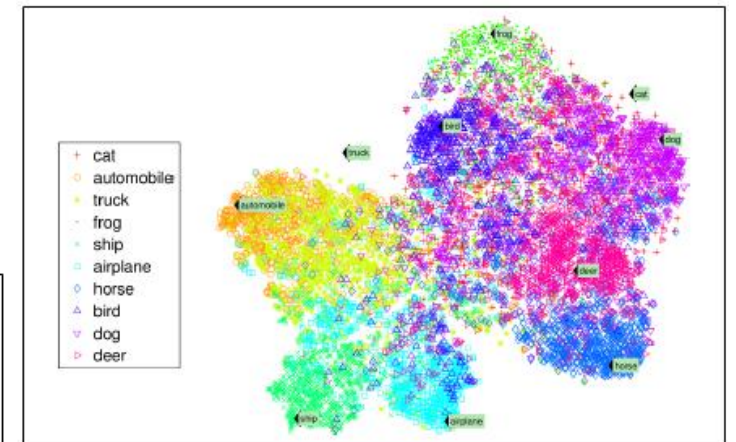
Unsupervised learning: Clustering



Unsupervised learning: Anomaly detection



Unsupervised learning: Visualization (t-SNE)



Hints for Model Selection

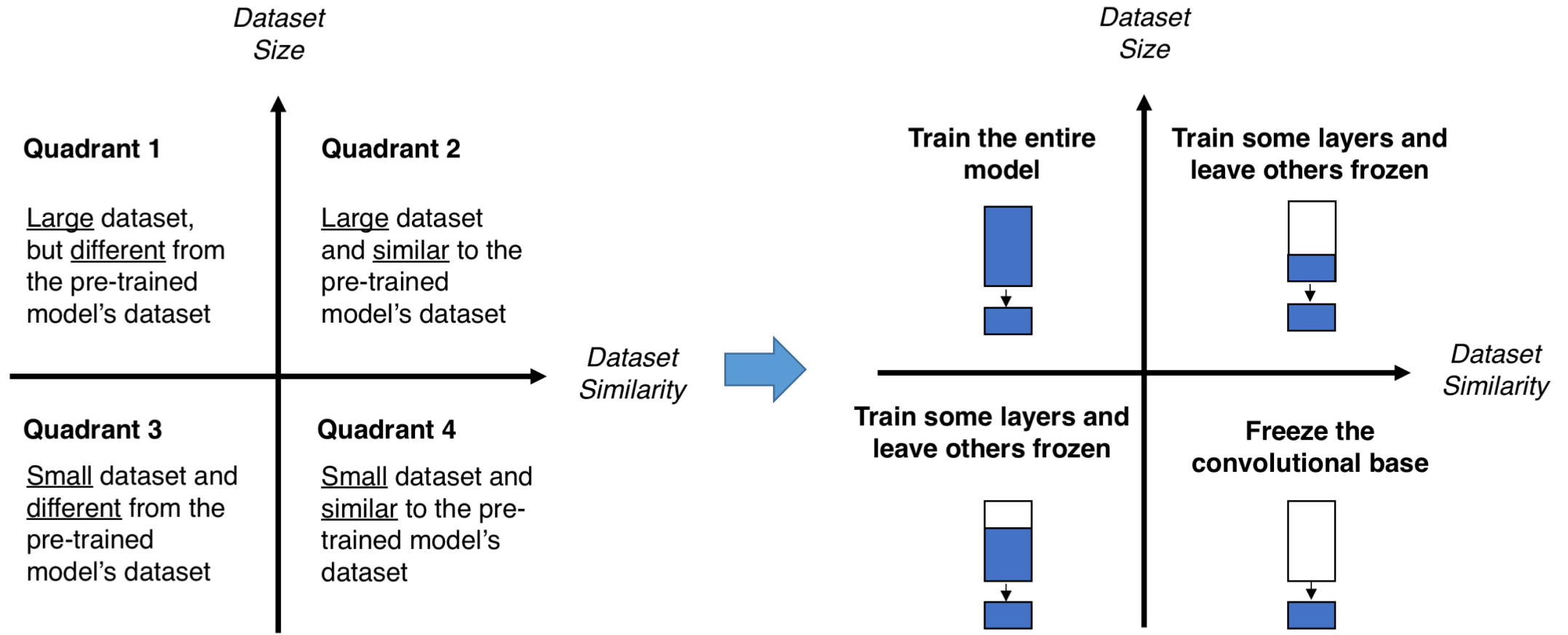
Application	Default Models
Value Prediction	Regression
Class Prediction	Classification
Little Data	“Classical” Models: Decision Trees, SVM, NN...
Little Labels, Anomaly Detection	Unsupervised Learning
Time Series Evaluation, Sensor Data	LSTM
Image Recognition, Computer Vision	CNN
Natural Language Processing	Transformer

In general **different models** should be tried, compared and combined!

Pretrained Models

- Instead of developing **algorithms**, one designs **architecture of deep learning NN**
- The architecture uses **well known modules** like Filters, Pooling, Inception, Residual Connections, SE Blocks etc
- Choice of modules and architectures based on **heuristics** according to the application area and the best DL praxis
- Applying Deep Learning:
 - Select **architecture** that best fit to your **application**
 - Used **pre-trained model** (transfer learning) if possible
 - **Learn only a few last layers** (the more new data available the more layers can be newly learnt)
 - **Change architecture only if necessary** or for experimental/research purposes

Transfer Learning: Use Pre-trained NN

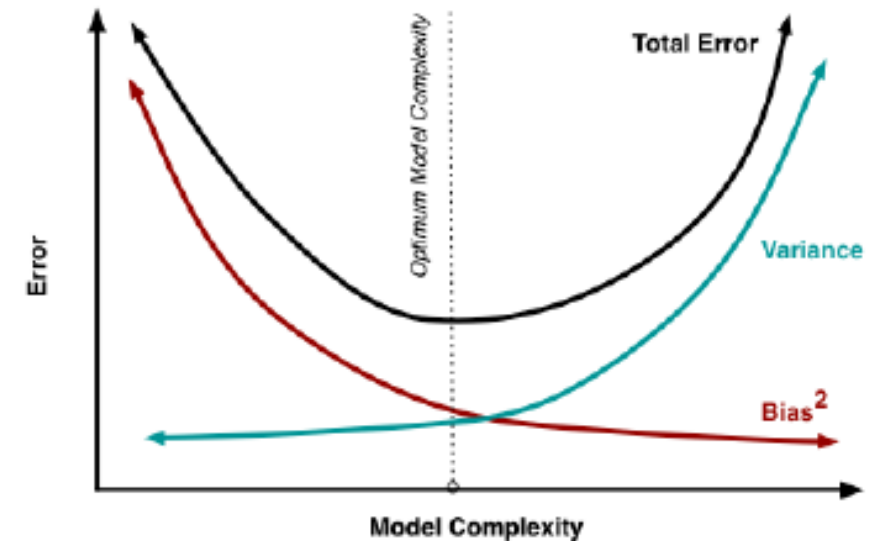


Shortlist Promising Models

1. Train many quick-and-dirty models from different categories (e.g., linear, naive Bayes, SVM, Random Forest, Neural Net, etc.) using standard parameters.
2. Measure and compare their performance.
 - For each model, use N-fold cross-validation and compute the mean and standard deviation of the performance measure on the N folds.
3. Analyze the most significant variables for each algorithm.
4. Analyze the types of errors the models make.
 - What data would a human have used to avoid these errors?
5. Perform a quick round of feature selection and engineering.
6. Perform one or two more quick iterations of the five previous steps.
7. Shortlist the top three to five most promising models, preferring models that make different types of errors.
8. Try to **combine the models that perform best**. The group (or “ensemble”) will often perform better than the best individual model

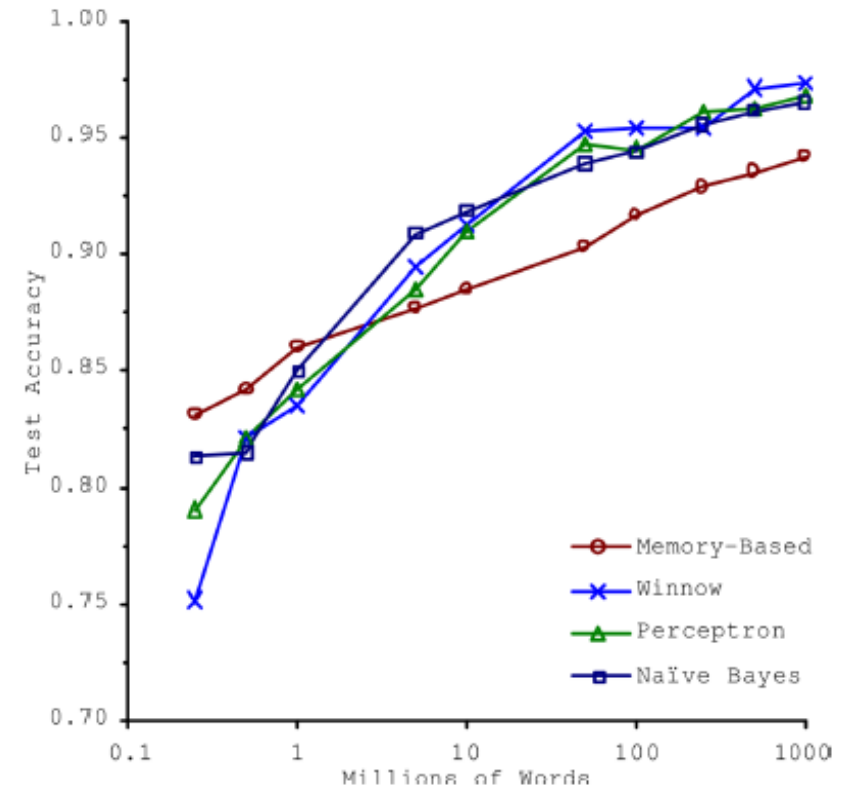
Bias-Variance Tradeoff

1. If performance bad on training set
 - **Underfitting** (high bias): too simple model -> increase model complexity
 - Problems with data: Outliers, false or insufficient features etc.
2. If performance good on training set and bad on validation/test set:
 - **Overfitting** (high variance): too complex model -> reduce model complexity
 - Regularization: Constrain model to improve generalization
 - More data and/or more representative data needed



The Unreasonable Effectiveness of Data

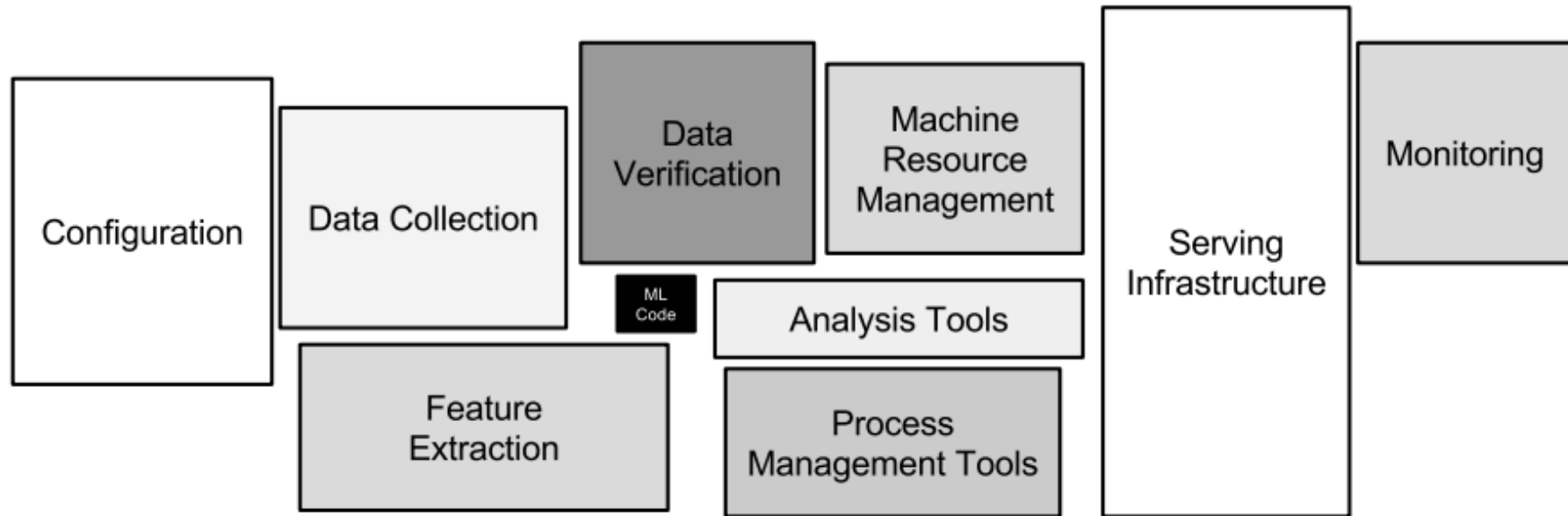
- Different Machine Learning algorithms, including fairly simple ones, performed almost identically well on a complex problem of natural language disambiguation once they were given enough data
- Peter Norvig, Director of research at Google and author of Artificial Intelligence: A Modern Approach, said: “***More data beats clever algorithms, but better data beats more data.***”



Michele Banko and Eric Brill, “Scaling to Very Very Large Corpora for Natural Language Disambiguation,” Proceedings of the 39th Annual Meeting of the Association for Computational Linguistics (2001): 26–33.

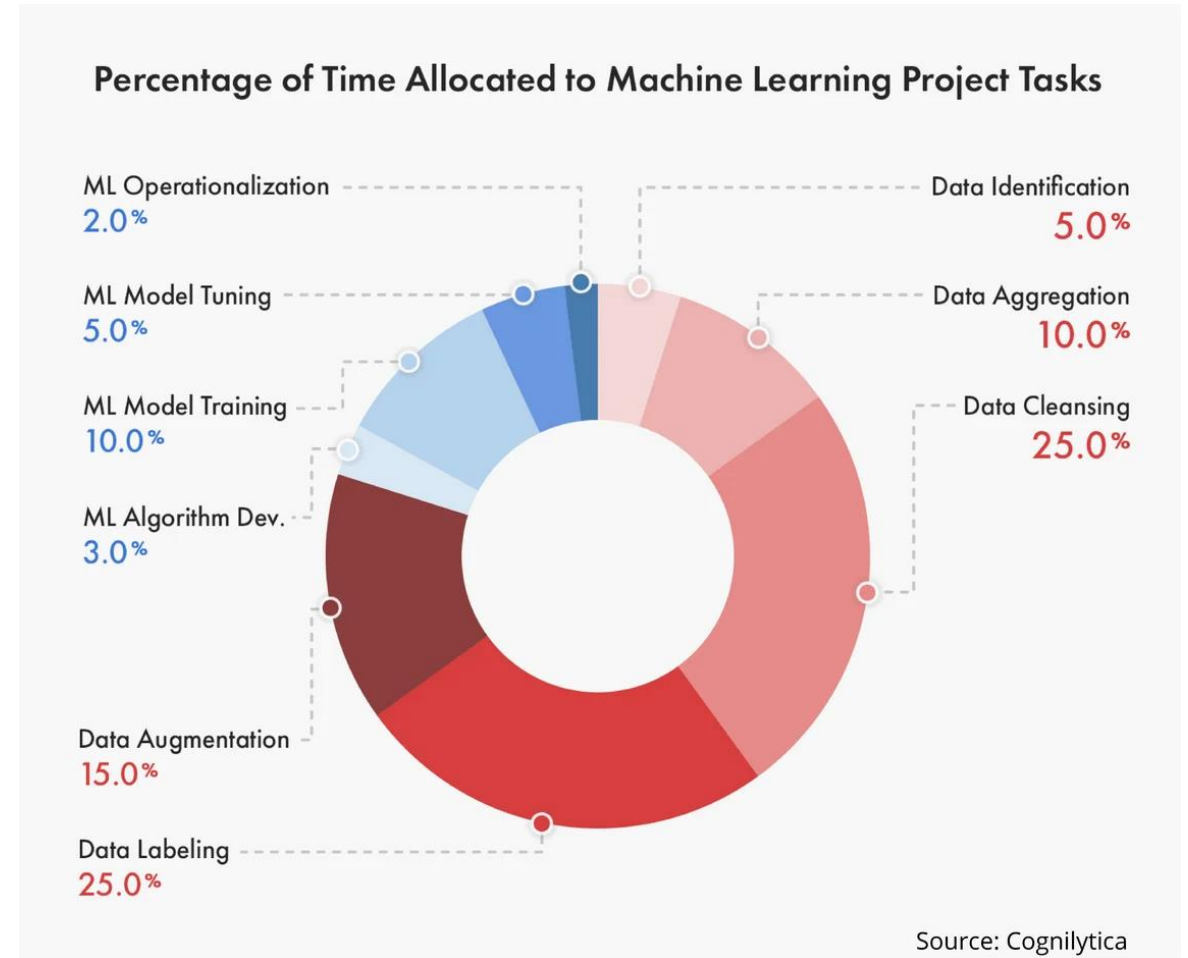
ML is only a part of much bigger system

- Only a small fraction of real-world ML systems is composed of the ML code, as shown by the small black box in the middle. The required surrounding infrastructure is vast and complex.



Effort per ML Project Task

- Data Acquisition and Annotation can take up to 50-80% of the total project time, particularly in scenarios where data needs are specific and no pre-labeled datasets are available.
- This percentage includes the time spent on cleaning and preprocessing the data, which are essential for making the data usable for machine learning models.



<https://www.cloudfactory.com/data-labeling-guide>

Present Your Solution

1. Document what you have done.
2. Create a nice presentation.
 - Make sure you highlight the big picture first.
3. Explain why your solution achieves the business objective.
4. Don't forget to present interesting points you noticed along the way.
 - Describe what worked and what did not.
 - List your assumptions and your system's limitations.
5. Ensure your key findings are communicated through beautiful visualizations or easy-to-remember statements (e.g., “the median income is the number-one predictor of housing prices”).

Launch!

1. Get your solution ready for production (plug into production data inputs, write unit tests, etc.).
2. Write monitoring code to check your system's live performance at regular intervals and trigger alerts when it drops.
3. Beware of slow degradation: models tend to “**rot**” as data evolves.
4. Measuring performance may require a human pipeline (e.g., via a crowdsourcing service).
5. Also monitor your inputs' quality (e.g., a malfunctioning sensor sending random values, or another team's output becoming stale). This is particularly important for online learning systems.
6. Retrain your models on a regular basis on fresh data (automate as much as possible).



Literature

- Géron, A.: *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*, 2022.
- Burkov, A.: *Machine learning engineering*, 2020.
- Huyen, C: *Designing Machine Learning Systems*, 2022.
- Taifi, M. *Clean machine learning code*, 2020.



Thank You For Your Attention